Federico Palatnik Moroz

Backend Software Engineer · C# since 2021 · .NET 8 and ASP.NET Core · Distributed Systems

· Buenos Aires, Argentina · linkedin.com/in/federico-moroz-1760a2241 · github.com/federicomoroz
· federicomoroz.github.io · case studies · Remote · UTC-3, full overlap with US business hours

PROFESSIONAL SUMMARY

Backend Software Engineer focused on **.NET 8 and ASP.NET Core**. Hexagonal architecture, with the rules verified by tests rather than by conventions: in my catalogue synchronisation service the same suite runs **27 contract cases against four interchangeable engines** — MySQL, PostgreSQL, SQLite and a hand-written in-memory one — and has already caught three engine divergences that do not surface when you test against one. Adding a fifth — **SQL Server**, say — is a provider project and its migrations, without touching a use case.

Distributed systems and messaging: WebSocket with backpressure, transactional outbox over Kafka, distributed rate limiting in Redis with a sliding window and an atomic Lua script. I write decisions down next to the alternative they displaced — eleven ADRs in the largest project, **two that came not from a meeting but from a failing test** and **one I corrected when my own benchmark refuted its original reasoning** —, plus the runbook and the wire protocol spec. Almost all of the code is published and auditable.

TECHNICAL SKILLS

.NET: .NET 8 · C# 12 · ASP.NET Core (MVC, Web API and minimal APIs) · Razor · Entity Framework Core (migrations, `AsNoTracking`) · LINQ · `async`/`await`, `CancellationToken` and `IAsyncEnumerable` · `BackgroundService` · `IHttpClientFactory` · dependency injection validated at startup · middleware · OpenAPI/Swagger · xUnit · `WebApplicationFactory` · Testcontainers · warnings as errors

Other languages: Python (FastAPI, SQLAlchemy) · Java 21 (Spring Boot, Spring Kafka) · TypeScript · Node.js · React · SQL

Data and infrastructure: MySQL · PostgreSQL · Redis · SQLite · Apache Kafka · Docker · Docker Compose · GitHub Actions · versioned migrations (EF Core, Flyway) · multi-stage Docker image running as non-root · container deployment (Render) · `live` / `ready` health checks

Architecture: SOLID · Clean Architecture / Hexagonal (ports & adapters) · MVC · Repository · Composition Root · command/query separation · Event-driven · Transactional Outbox · Idempotent consumer · Circuit breaker · Backpressure · WebSockets · SSE

Testing and operations: multi-engine contract tests · real MySQL and PostgreSQL in CI via Testcontainers · purpose-built load tests · structured logging with Serilog, with the credential masked in the access log · runbook and ADRs

PROFESSIONAL EXPERIENCE

Backend Developer

2022 – Present

Freelance · Remote

I design, build and deploy backend services end to end, and I write them the way a team's code gets written: ADRs recording what was rejected and why, an operations runbook, the wire protocol spec and one integration guide per type of consumer, and — in the Java service — architecture rules pinned by ArchUnit tests that fail the build in CI.

> **Nexo — B2B catalogue synchronisation** .NET 8 · ASP.NET Core MVC · EF Core · MySQL · PostgreSQL · Redis · WebSockets · SSE · OpenID Connect
> github.com/federicomoroz/nexo · case study
>
> An ERP publishes catalogue, prices and stock, and a reseller network consumes them over two transports on the same domain: HTTP for point queries and a WebSocket with a purpose-built frame protocol for bulk pulls, with per-batch acknowledgement backpressure. A seven-step authorization pipeline shared by both transports, with HMAC-SHA256 and a quota shared in Redis, and a Razor operations panel over SSE. **A test found that the `X-Forwarded-For` setup every sample copies — clearing `KnownProxies` and `KnownNetworks` — makes ASP.NET Core trust the header from anyone**: the per-key IP allowlist could be bypassed with a header, and the comment I had written next to it asserted the opposite of what the framework actually did.
>
> Another test found a handler that had never been registered in the container: `POST /v1/catalog` returned 500, and the error would have surfaced on the ERP's first batch. The fix was not registering it but **validating the container at build time** —

`ValidateOnBuild`, `ValidateScopes` and `AddControllersAsServices`, so the validation reaches the controllers too — so an unregistered dependency stops the process from starting instead of failing on the first request.

Measured, not estimated: pulling 48,000 items costs **1 authorization instead of 96** against the same work over paginated HTTP; and in a separate run against 8,000 items, 40 simultaneous connections all complete the pull while an ERP change reaches every one at **p50 68 ms**, and that latency **does not move between 1 and 40 connections**: the cost is in the write path, not the fan-out. **356 tests** in CI — 60 domain, 93 API, 95 application and 108 contract — with MySQL and PostgreSQL in real containers.

The application layer **does not reference a single EF Core package**, and the fourth provider — in memory, hand-written — is what proves it: if the ports leaked anything, it would not compile. PostgreSQL was added last — a provider project, its migrations, a four-line class — and all the contract cases passed on the first run. A separate CI job **fails the build if anyone changes the model without generating the migration**.

Shipping Quote .NET — hexagonal rate quoter .NET 8 · ASP.NET Core · EF Core · MySQL · Testcontainers
github.com/federicomoroz/shipping-quote-dotnet · case study

The same quoter I wrote in Python, ported to ASP.NET Core to separate architecture from language habit. Async end to end with no `.Result` anywhere, a `CancellationToken` threaded down to the adapter, and `CreateLinkedTokenSource` so the contract timeout does not override the incoming request's cancellation. The three carriers — simulated — are queried with `Task.WhenAll`, and **a test measures the clock and fails past 700 ms**: in parallel it is 306, sequential would be 900. A middleware maps domain exceptions to HTTP status codes. **63 tests** (50 unit + 13 integration against a real MySQL).

Order Outbox Service Java 21 · Spring Boot · Kafka · PostgreSQL
github.com/federicomoroz/order-outbox-service · case study

Dual write solved with a transactional outbox: the order and its event are written in a single Postgres commit and a separate relay publishes them to Kafka with backoff, so a broker outage cannot take the order down with it. The consumer dedupes by `event_id`, turning the broker's at-least-once into exactly-once at the business level. **86 tests** (72 unit + 14 integration) and ten ArchUnit.

AI training plan generator Flask · Claude API · SSE · Docker
federicomoroz.github.io/en/projects/mega-training-system/

A platform in daily production use by a client: SSE streaming, a circuit breaker around the model calls, and idempotent generation.

Backend Developer / Automation Engineer

Nov 2024 – Mar 2026

Del Mundo a Tus Manos · International e-commerce and logistics · In-house, full-time

I designed and maintained the operations platform of an international e-commerce business on Airtable, working alongside the operations team that used it every day: a data model of 11 linked tables, ~115,000 records and 48 automations, with the operator-facing interfaces built on top in Node.js, React and TypeScript on the Interface Extensions SDK, and integrations with Amazon plus Mercado Libre and Tienda Nube (LatAm e-commerce platforms), the national tax authority API, international couriers and a Laravel/PHP backend, handling authentication, secrets management and IP allowlisting. **I cut operational processing time from approximately 10 minutes to 5 seconds across ~2,100 monthly transactions.** I diagnosed and fixed a silent failure on batches of 100+ records — requests hung with no trace in the execution log — with an AbortController, retry with exponential backoff and non-blocking per-record error handling.

Game Developer and Programming Instructor

2021 – 2024

Freelance

Combat systems and tooling on commission in **C#** on Unity and Unreal —a published turn-based battle engine—: this is where my C# starts, where I applied Flyweight, Strategy and Chain of Responsibility to concrete performance problems, and where I learned to work against real-time constraints — a per-frame budget, nothing allowed to block the main thread — the same reasoning I later carried into asynchronous I/O and backpressure. Alongside that I taught programming one to one, doing **code review** on each student's code.

EDUCATION AND LANGUAGES

Education: Escuela Da Vinci — Programming (2021 – 2023) · UADE — Marketing (2007 – 2010)

Languages: Spanish — Native · English — Full Professional Proficiency (C2)